



第四章 CPU和指令集架构

XSUN@GZHU

2026/05/24

第一、二章我们学习了组成计算机系统的基本组件：运算与存储。第三章我们学习了如何将人类易懂的汇编语言转化为计算机懂的机器码，进而通过硬布线或微程序控制的方式产生正确的硬件控制信号。这一章我们将以**设计一个最小可编程的CPU**为目标，学习处理更加复杂的汇编语句，学习CPU和指令集架构设计的相关知识。

所谓CPU，即Central Processing Unit-中央处理器，他的一生都在**重复**做着两件事情：**取(fetch)指令和执行(execute)指令**，也因此我们把“取指令和执行指令”的时间合在一起，称为CPU的一个**指令周期 (Instruction Cycle)**。

所以，对于本章，我们首先需要明白什么是指令(Instruction)，然后理解如何取和如何执行，这样我们就可以完全理解CPU进而设计自己的CPU。

指令集架构-Instruction Set Architecture (ISA)

为了更加明确我们的设计目标，我们需要首先设计最终CPU所能具有的“功能”，即这个CPU能够执行怎样的硬件指令。这也同时决定了我们汇编语言所支持的语法(syntax)，因为软件和硬件是需要做到适配和统一的。

寻址方式-Addressing Mode

在上一章中，我们使用如 `MOV A 3` 这种汇编代码来表示将寄存器A中的值**读**出来并**写**入到RAM地址为3处。但是如果我想表示把十进制数值3写入到RAM地址为3处呢？为了拥有更好的human-friendly特性，我们需要给用户提供更好编程体验，因此我们应当支持尽可能多的编程需要。我们可以用 `MOV 3, [3]` 这种汇编语法来表达上述需求。因此，我们定义如下寻址方式：

- **立即数**：将数字5写入寄存器A

```
MOV A, 5
```

- **寄存器寻址**：将寄存器B中的值写入寄存器A

```
MOV A, B
```

- **直接寻址**：将内存RAM地址5处的值写入寄存器A

```
MOV A, [5]
```

- 寄存器间接寻址：将内存RAM地址等于寄存器B中的值处的值写入寄存器A

```
MOV A, [B]
```

当然，诸如 `MOV 5, A` 这种汇编语句是非法的，因为对于 `MOV` 指令来说，目的操作数必须是可写入的，一个立即数是无法写入的。

注意：这里我使用了x86的对于目的操作数和源操作数的形式规定，即目的操作数在左，源操作数在右。

图灵完备-Turing Complete

所设计的CPU，除了具有基本的算术和逻辑运算功能外，还需要使其是图灵完备(Turing Complete)的，即需要支持条件判断和循环。事实上，在汇编代码中，我们可以通过和**跳转**来实现**条件判断**和**循环**。跳转的硬件本质即是对程序计数器PC写入一个数

($PC-CS=1, PC-WE=1, PC-EN=0$)，这个数代表了程序需要跳转到的具体位置。在汇编代码中我们可以使用跳转指令 `JMP` 来实现跳转，比如：

```
MOV D, 1
increase:
    ADD D, 1
    JMP increase
```

以上汇编程序会一直使得D寄存器中的值不断加1，其本质是一个死循环（你看我们已经使用 `JMP` 实现了循环）。这里的 `JMP` 成为**无条件跳转**，其后紧跟的是具体要跳转到的位置，也就是 `increase` 所标注的位置，这里的 `increase` 被称为**标签(label)**。这也是C语言中 `goto` 语句的汇编实现：

```
goto label;
...
label: statement;
```

`JMP` 是无条件跳转，我们需要有条件跳转才能实现条件判断。

- 条件判断

```

CMP D, 5
JO case
MOV A, 1
JMP end
case:
    MOV A, 0
end:
    HLT

```

这里的 `CMP D, 5` 是使用ALU的减法来实现的，但是和 `SUB D, 5` 不同的是，`CMP` 不会把ALU的结果写入到任何一个寄存器或RAM中，只会获取ALU的程序状态字(program state word-PSW)。而指令 `JO` 的意思就是当PSW中的 `overflow` 标志位（溢出位）为1时执行跳转（在硬件上就是将紧跟 `JO` 后面的数值写入到PC中），否则忽略该句，直接执行下一句。

由此，可分析上述代码的功能：如果D寄存器的值大于等于5，则把A寄存器的值设置为1，否则把A寄存器的值设置为0。思考：可否删除end标签？

- 循环

接着，我们来用条件跳转来实现不是死循环的循环：

```

MOV D, 1
increase:
    INC D
    CMP D, 5
    JO increase
    HLT

```

这里的 `INC` 是自增1指令（类似C语言 `x++`），利用的是ALU的加1功能。同理，我们也有 `DEC` 指令（`x--`）。这里的 `HLT` 是停机指令，也就是关闭CPU的时钟源，CPU的各硬件电路的状态不再变化。所以，上述代码的功能，就是将寄存器D中的值初始化为1后，然后不断加一，直到其值等于6（究竟=5还是=6，自行分析）。

上述两段代码中的 `JO` 就是有条件跳转，这个条件的判断源就是ALU的PSW输出。类似地，我们还可以定义 `JZ` 也就是，PSW的输出ZERO标志位置位(等于1)时执行跳转。当然，还可以定义 `JNZ, JNO`，也即对应标志位等于0时执行跳转。

指令集架构设计

根据上述代码来归纳总结下一个图灵完备的CPU应当支持的最基本的指令

有： `MOV, ADD, SUB, INC, DEC, AND, OR, XOR, NOT, CMP, JZ, JO, HLT` 等。另外我们发现有些指

令后面有两个操作数(如: `MOV,ADD,CMP`), 有些有一个(如: `DEC,JMP`), 有些没有(如: `HLT`), 而对于有操作数的指令, 我们还要指定其寻址方式。对于一个8bit的CPU来说, 我们需要一个8位二进制说来区分上述所有情况, 比如如何区分 `MOV [2], [4]` 和 `MOV [2], 4`, 虽然都是 `MOV`, 而且目的操作数都为2, 源操作数都为4,但是我們对于后面数字的解析式完全不同的。为此, 我们可以进行如下的设计:

- 二地址指令 (有2个操作数的指令) - 共7个

7	6:4	3:2	1:0
1	OP	DST_AM	SRC_AM

OP取值: `MOV=0,ADD=1,SUB=2,CMP=3,AND=4,OR=5,XOR=6`

- 一地址指令 (有1个操作数的指令) - 共8个

7	6	5:2	1:0
0	1	OP	DST_AM

OP取值: `INC=0,DEC=1,NOT=2,JMP=3,JO=4,JNO=5,JZ=6,JNZ=7`

- 零地址指令 (没有操作数的指令) - 共1个

7	6	5:0
0	0	OP

OP取值: `HLT=0b111111`

其中, OP表示操作码, 区分的是具体的功能 (比如是做加法还是减法), AM表示address mode, 也就是寻址方式:

- `0b00` 立即数
- `0b01` 寄存器寻址
- `0b10` 直接寻址
- `0b11` 寄存器间接寻址

DST_AM就是目的操作数 (destination operand) 的寻址方式而SRC_AM就是源操作数 (source operand) 的寻址方式。根据上述规则, 我们便可以把汇编代码转化成二进制机器码, 完成汇编代码的编译过程, 如:

汇编代码	机器码	说明
MOV C, 4	0b10000100 (0x84)	DST=C, 寄存器寻址; SRC=4, 立即数
MOV [C], 4	0b10001100 (0x8C)	DST=C, 寄存器间接寻址; SRC=4, 立即数
AND C, D	0b11000101 (0xC5)	DST=C, 寄存器寻址; SRC=D, 寄存器寻址
DEC [5]	0b01000110 (0x46)	DST=5, 直接寻址
JZ 5	0b01011000 (0x58)	DST=5, 立即数

这样，我们就完成了指令系统的设计。不过，你可能会问，虽然我们用了个8位二进制数表示了我们的指令，那目的操作数和源操作数该如何表示呢？这其实涉及到我们该如何开发一个汇编编译器的问题。

汇编器 Assembler

事实上，对于一个8bit CPU来说，除了指令集中的指令是用8位二进制表示的之外，目的操作数和源操作数也是各用一个8位二进制数来表示的，CPU内会有专门的寄存器来存储操作码、目的操作数和源操作数。例如，对于 MOV C, 4 这句汇编代码，其整过汇编编译后的完整二进制应为3个字节：10000100 00001000 00000100 [B]，其中第一个字节即为二地址指令MOV在目的操作数是寄存器寻址，源操作时是立即数寻址时对应的指令码，而第二个字节是表示C寄存器的编码（这与硬件结构有关，这里等于8只是举例），而第三个字节即为源操作数的数值，应为4，对应8bit二进制为 0b00000100。同理，MOV [C], 4 经过编译后应为：10000100 00001000 00000100 [B]，其只和 MOV C, 4 的编译结果中的操作码有区别。

对于一地址和零地址指令，为了在形式上统一，我们也用3个字节来编码一句汇编代码，把没的数补0即可，如 DEC [5] 经过编译后为 01000110 00000101 00000000[B]，而 HLT 则为 00111111 00000000 00000000[B]。

因此，对于如下汇编代码文件：

```
MOV C, 4
FLAG1:
    CMP C, 5
    JNZ END
    MOV C, 4
END:
    HLT
```

经过汇编编译器后的输出应为：

```
[hex][8bit Data]:
ADDR: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E
DATA: 84 08 04 B4 08 05 5C 0C 00 84 08 04 3F 00 00
```

注意 0x5C 对应 JNZ，其后的值是 0x0C，指向的正是 HLT 对应的机器码 0x3F。将这段机器码载入（实际中是由操作系统完成，但实验中我们手动加载）到计算机的内存RAM中，CPU便可以进行读取和执行。

取指令-fetch instruction

二进制机器码加载到内存RAM后，CPU如何进行读取和执行呢？我们分析目前的指令集可以发现，有些指令的功能无法在一个时钟周期（一个上升沿和一个下降沿）内完成，比如 ADD [2], C，假设在某硬件（单数据总线，ALU的输入A和B分别接着A和B两个寄存器）上执行这个指令，需要：

1. 第1个时钟周期：将内存地址为2处的值读出并写入寄存器A
2. 第2个时钟周期：将寄存器C的值读出写入寄存器B
3. 第3个时钟周期：将ALU的输出值写入到内存地址为2处

如此，便需要CPU在3个周期内都保持在执行这个指令的状态，也就是说我们需要暂存和锁定这个指令，为此，我们需要一个寄存器专门保存当前所需要的指令，这个寄存器称为**指令寄存器(IR: Instruction Register)**。

此外，我们可以看到指令集中的操作数具有多种不同的寻址方式，有时我们需要的是操作数本身的数值，有时需要的是操作数数值所对应的寄存器或内存空间，因此，为了设计上的灵活，我们也需要两个寄存器来分别暂存目的和源操作数的值，这就是**目的操作数寄存器 DST-R**和**源操作数寄存器 SRC-R**。然后，为了通过单条总线实现对RAM的稳定灵活的访问，我们还需要一个寄存器专门保存目前要访问的内存地址的值，这就是**内存地址寄存器MAR(Memory Address**

Register)。

PS: 对于IR, MAR, DST-R, SRC-R这些寄存器存在的必要性, 如果现在难以理解也属于正常, 你会在设计和实现自己的CPU的过程中慢慢有所体会的。

因此, 所谓的CPU取指令周期, 就是把操作码(opcode)写入IR, 把操作数(operand)写入到对应的寄存器中。但是, CPU如何知道从内存中哪个地址加载操作码和操作数呢? 对! 我们可以使用程序计数器PC, 用PC的值来帮我们指向正确的内存地址。

根据上一节的内容, 我们知道, 一条汇编语句对应连续的3个字节数据, 如果计算机的RAM是8位地址位8位数据位的话, 那么加载一条汇编语句, 8位的PC需要自增3次。因此, 一个取指令周期对应的时钟周期数为6:

1. 第1个时钟周期: 读取PC的值, 写入到MAR
2. 第2个时钟周期: 读取RAM的值, 写入到IR, PC自增1
3. 第3个时钟周期: 读取PC的值, 写入到MAR
4. 第4个时钟周期: 读取RAM的值, 写入到DST-R, PC自增1
5. 第5个时钟周期: 读取PC的值, 写入到MAR
6. 第6个时钟周期: 读取RAM的值, 写入到SRC-R, PC自增1

这样, CPU便把内存中的指令(操作码 + 操作数)加载到了相应的寄存器中, 下一步就是执行这个指令: 即利用这些寄存器中的数值去产生正确的硬件控制信号, 从而完成指令所期待的功能。

执行指令-execute instruction

我们知道, 执行指令的本质是给硬件部件提供正确的控制信号, 因此CPU硬件电路的基本功能是要将已经加载到IR,DST-R和SRC-R寄存器中的值转化成正确的控制信号, 本设计我们采用微程序控制, 因此, 也就是要产生正确的微程序存储器的地址信号。

我们先来分析一下, 这个微程序存储器的地址信号应当包含哪些信息? 首先, IR寄存器中的值是操作码, 决定了此次操作的具体功能, 因此需要。然后, 条件跳转指令 `jz, jo` 等还需要此时ALU的PSW的值, 因此我们也需要把PSW的值也包含在内。最后, 我们知道执行不同的指令所需要的时钟周期是不一样的, 比如 `MOV C, 4` 只需要一个时钟周期(SRC-R为读状态, 寄存器C为写状态), 而我们上面所举得例子 `ADD [2] C` 则需要3个时钟周期, 且每个周期内的硬件控制信号都是不同的, 因此为了让硬件电路依次产生正确的控制信号, 我们也需要把时钟周期的数值包含在所要产生的地址信号中。综上, 我们对**微程序存储器的地址信号**的组成作如下定义:

15:8 (共8位)	7:4 (共4位)	3:0 (共4位)
IR	PSW	CYC

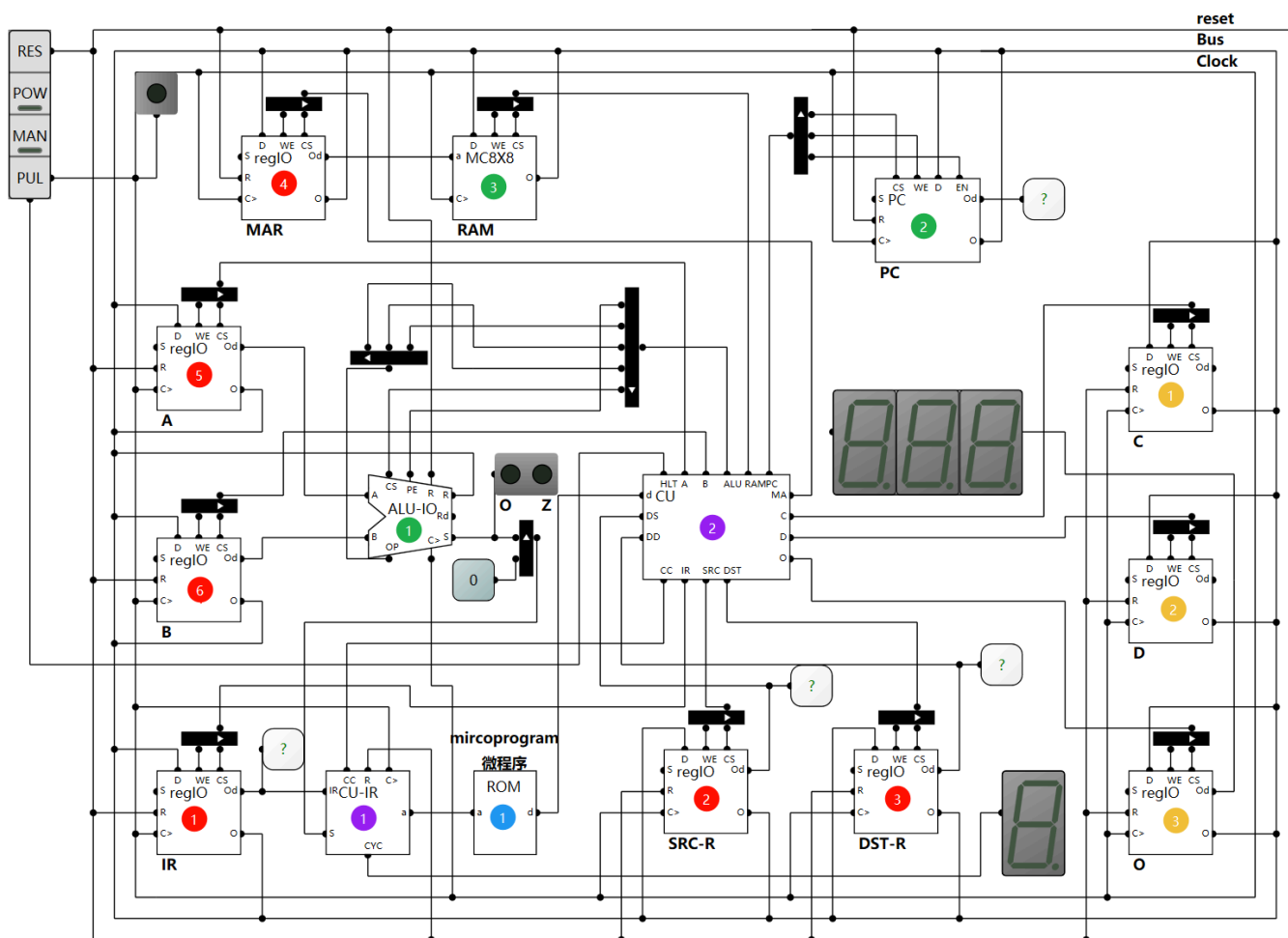
- IR: 指令寄存器中的值
- PSW: 程序状态字, 来自ALU
- CYC: 时钟周期, 最大为 $2^4=16$ 个时钟周期

CYC需要在执行完一次控制后自动+1, 因此可以用一个程序计数器PC来实现。但CYC不会一直自增1, 当前指令完成时, CYC应当清零, 开启下一个指令的周期。因此微程序中需要有一个额外的控制信号, 来通知把对CYC进行计数的PC清零, 从而进入下一个取指令操作。那么该如何设计我们的微程序呢?

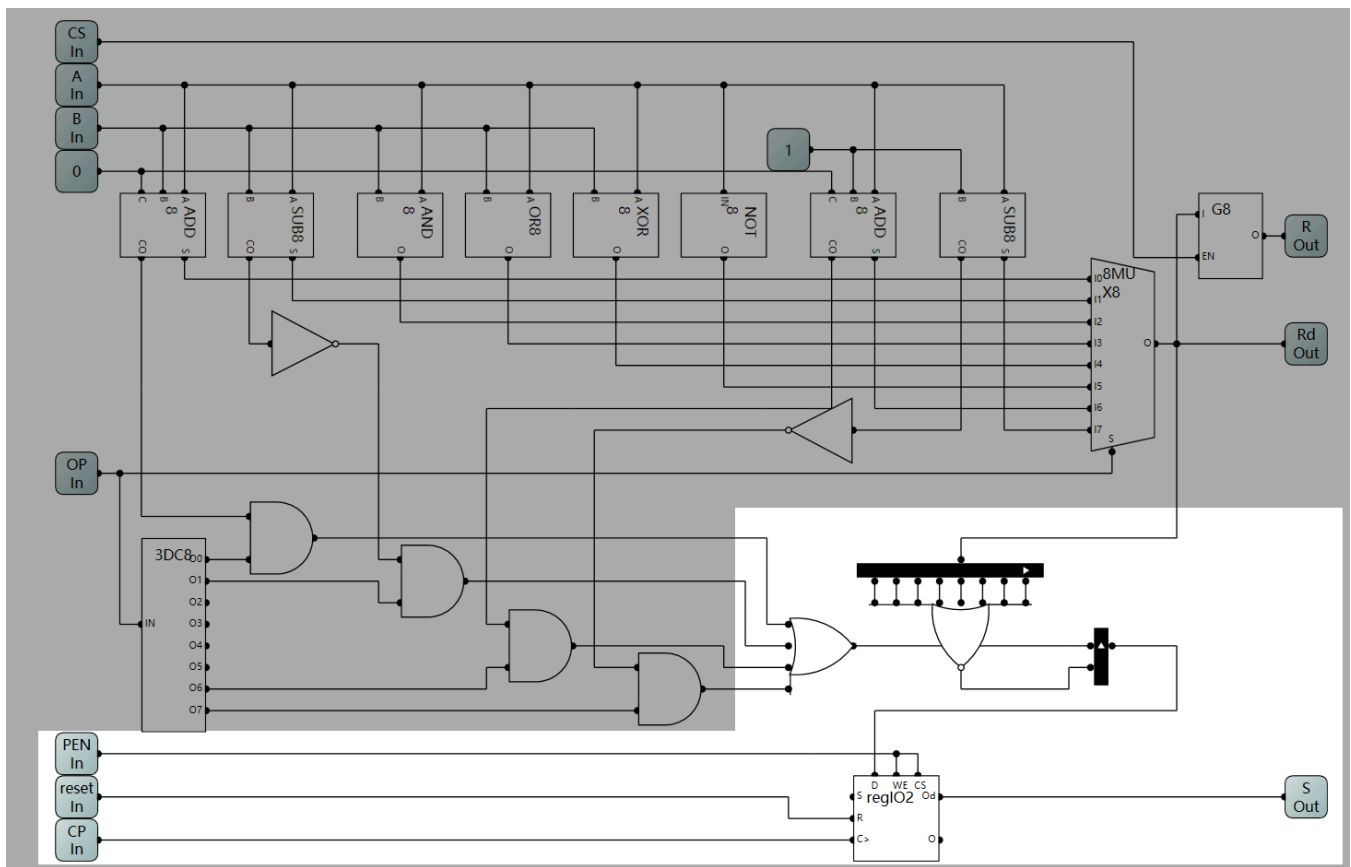
CPU硬件架构

因为微程序是和硬件紧密相连的, 要设计微程序, 就需要先把硬件固定下来。我们已经完成了指令集的设计, 也明确了CPU的功能, 根据上述分析, 我们的CPU硬件结构需要包括以下单元:

1. 专用寄存器组: IR, SRC-C, DST-R, MAR, A, B
2. 通用寄存器组: C, D, O
3. ALU (输入端接A,B寄存器)
4. RAM(RAM控制器)
5. PC
6. 微程序存储器ROM
7. CU (控制单元, 用于产生硬件控制信号)

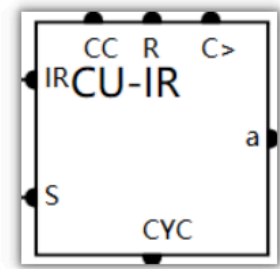
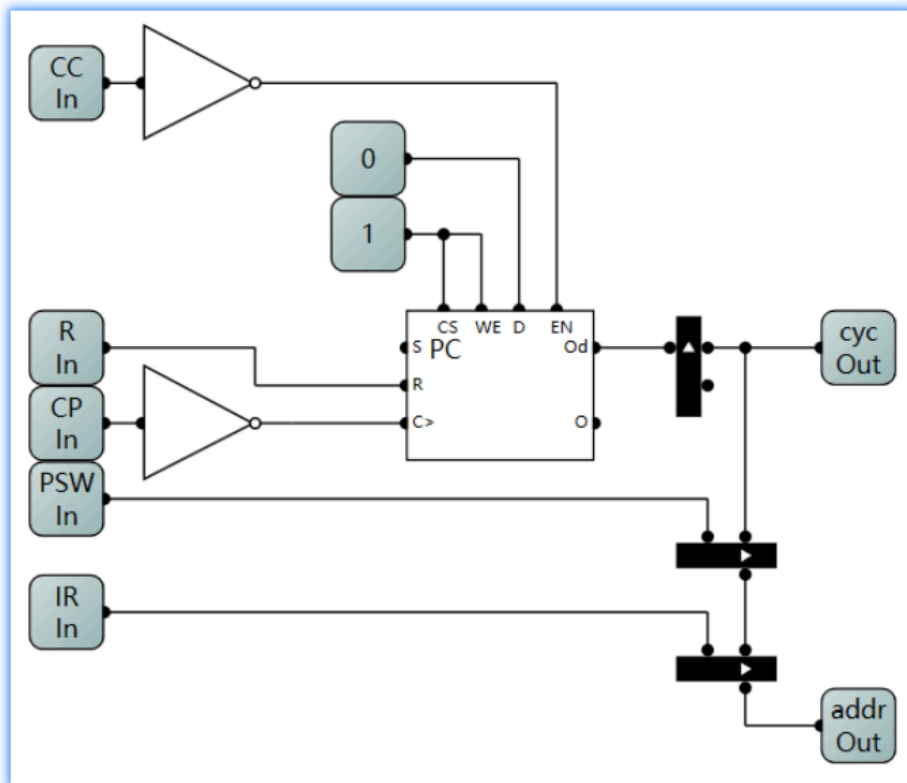


其中1-6是前三章的内容，把他们接到总线上即可。但是这里的ALU需要进行一些更新，原因是：条件跳转指令需要获得PSW的状态从而决定是否发生程序跳转(将跳转指令后面的操作数的值写入PC中)，而当IR中为跳转指令时，ALU并不处于工作状态，因此ALU需要暂存上一次运算的PSW，从而方便跳转指令读取。为实现暂存PSW的功能，我们需要给ALU内部加一个寄存器（再次体会寄存器为什么叫做“寄存”器），由于我们的PSW只有两位，所以一个2位的寄存器即可：



注意 `PEN` 信号控制了PSW寄存器的 `CS` 和 `WE` 信号，也就是说当 `PEN=1` 时，需要将ALU的状态标志位写入到寄存器中。事实上，我们除了在执行 `CMP` 指令时，会单独让 `PEN=1` 外，其他时候只要我们使用了ALU进行计算，都会使 `PEN=1` 以使其更新PSW的输出。

本章需要设计的是控制单元，其有两个部分，一是CU-IR用于产生访问微程序存储ROM的16位地址：



它的内部是一个PC，地址输出 `addr` 是按照规则把IR,PSW和CYC进行拼接而成的。CYC输出在电路中接了一个数码管，用于监视指令周期。

控制单元的另一部分根据微程序存储器的输出 `d` 产生硬件控制信号。输出 `d` 是何种形式属于微程序设计的范畴。

微程序设计

分析CPU硬件电路可知，需要控制读写的存储单元总个数为 $10 = 6 + 3 + 1$ ，因为 $2^3 < 10 < 2^4$ ，因此我们需要4位二进制数来表示究竟读写的是哪一个硬件，又因为我们有目的和源操作数两个，因此就需要共8位来表示。控制PC需要3位（`CS`，`WE`，`EN`），控制ALU需要5位（`CS`，`OP`，`PSW-EN`），1位产生CYC的清零信号，1位产生停机信号。

PS: ALU的 `PSW-EN` 信号是新增的，原因是我们把PSW信号包含到了访问微程序ROM的地址中，在不需要PSW信号的时我们不希望PSW信号的值影响。

除此之外，考虑：`MOV C, D`、`MOV [8], D` 和 `MOV [C], D` 这3个指令：在汇编编译后，其目的和源操作数均为8和9,但其意义却很不同，对于 `MOV C, D` 来说，`DST-R`中的值所指示的C寄存器应处于写状态（`DST-R-Write`）；而对于 `MOV [C], D` 来说，`DST-R`中的值所指示的C寄存器处于读状态（`DST-R-Read`），其值要读到MAR中，而真正要写的是RAM；对于 `MOV [22], D` 来说，需

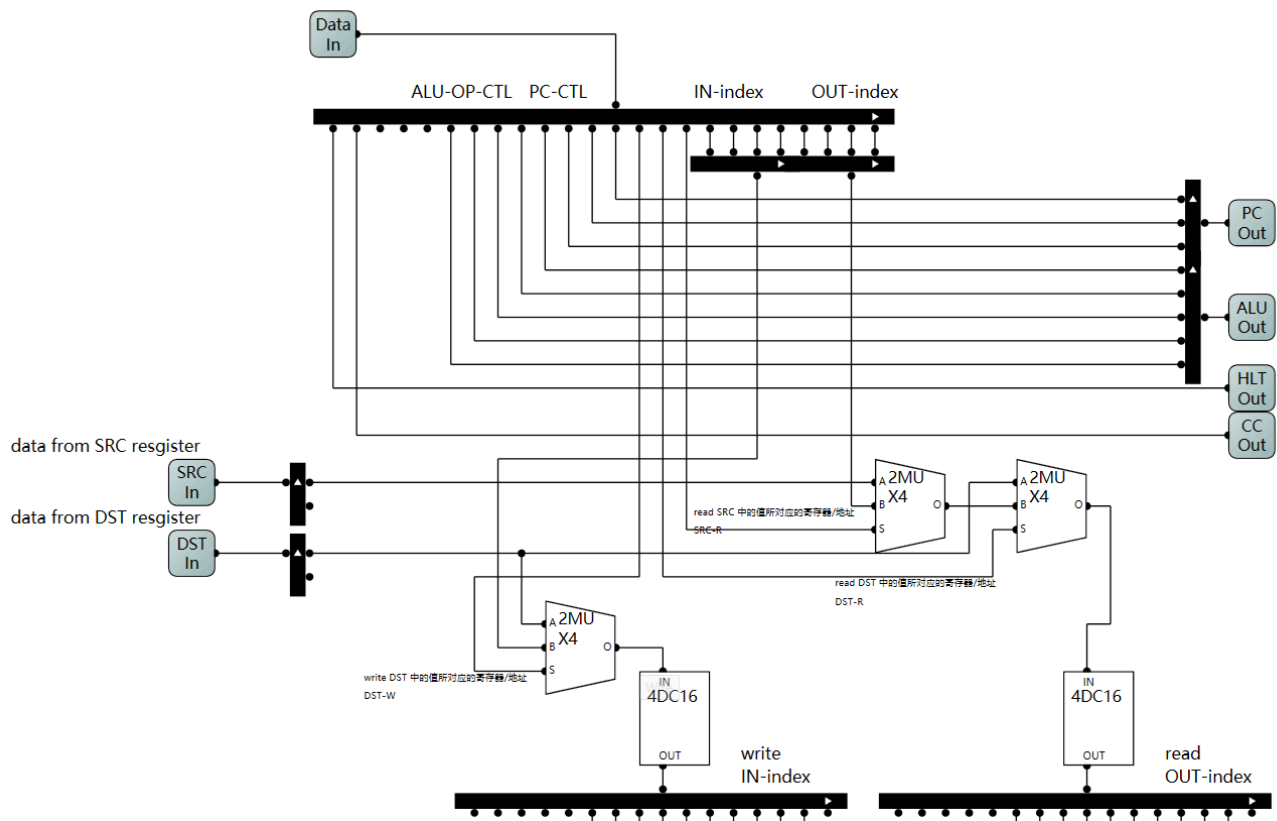
要直接把DST-R中的值输出（DST-R-Out）。相似地，对于SRC-R来说，MOV C, D 和 MOV C, [D] 都意味着SRC-R中的值所指示的D寄存器处于读状态（SRC-R-Read），而 MOV C, 9 需要直接把SRC-R中的值输出（SRC-R-Out）。SRC-R-Out 和 DST-R-Out 是和其他寄存器具有一样的控制逻辑，因此只需要再增加 DST-R-Write、DST-R-Read 和 SRC-R-Read 这3位控制信号即可。

综上所述，给出微程序存储器的输出d（共24位）的定义：

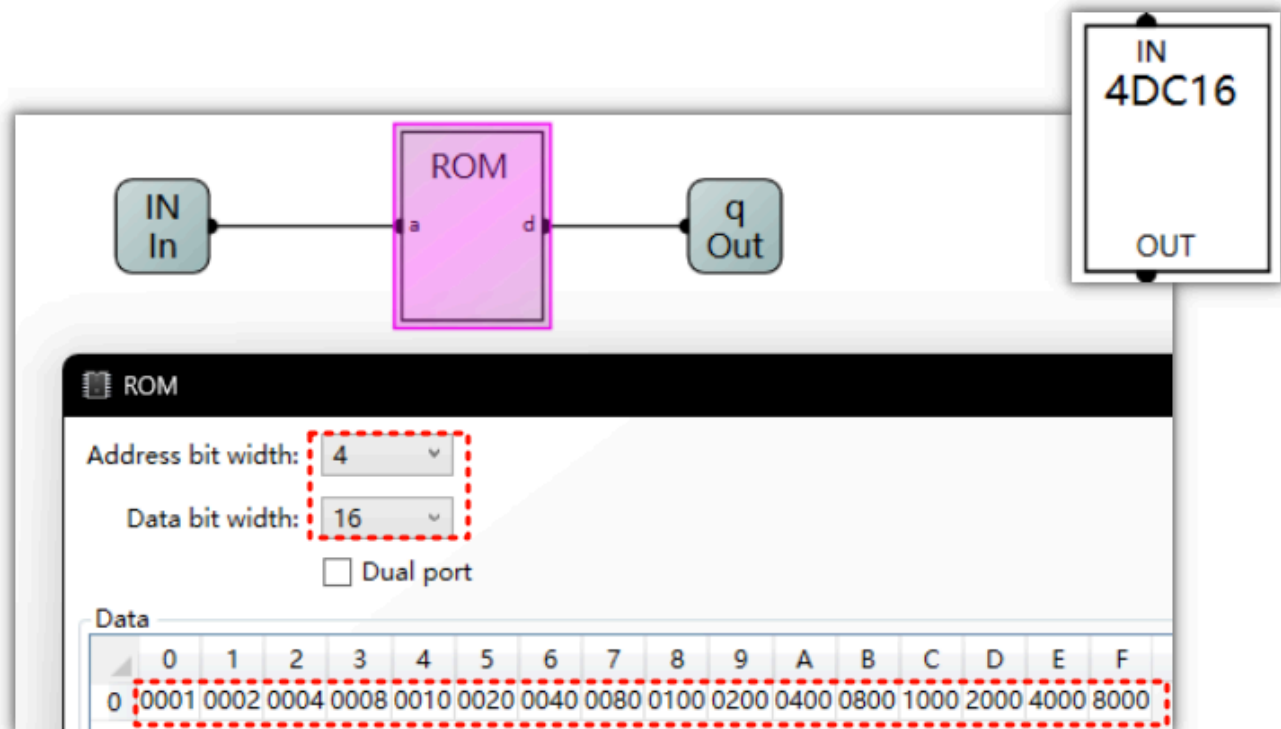
23	22	21:19	18:14	13:11	10	9	8	7-4	3-0
HLT	CYC_CLR	-	ALU	PC	DST_W	DST_R	SRC_R	W	R

- R: 需要读的寄存器索引值
- W: 需要写的寄存器索引值
- SRC_R: 读 SRC-R 中的值所对应的寄存器/RAM地址
- DST_R: 读 DST-R 中的值所对应的寄存器/RAM地址
- DST_W: 写 DST-R 中的值所对应的寄存器/RAM地址
- ALU: ALU-CS, ALU-OP, ALU-PSW-EN
- PC: PC-CS, PC-WE, PC-EN
- CYC_CLR: 指令周期计数器清零
- HLT: 停机，本质是关闭 clock 的输出

根据上述规则，便可以给出此部分控制单元的硬件实现（下方寄存器控制输出已省略）：



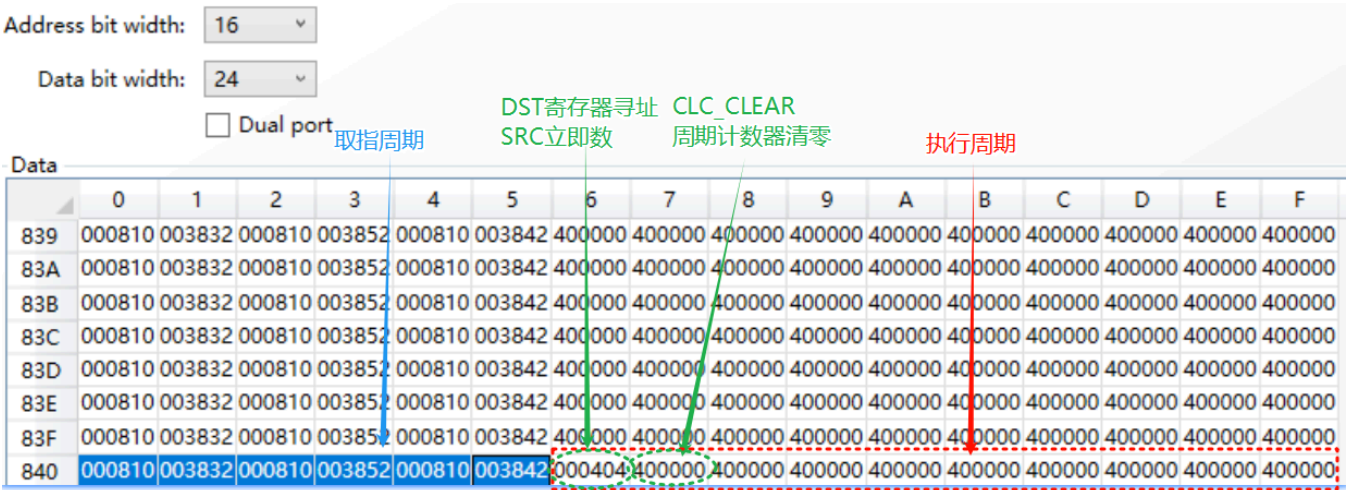
上图中 4DC16 是4-16译码器，实现原理类似3-8译码器，但这里用的是ROM实现：



如此，我们只需要将需要读写的寄存器编号分别送到两个4-16编码器的输入端口，就可以产生只有那个编号对应位置为1，其余为0的输出，进而组合成对应寄存器的 CS 和 WE 信号。

微程序ROM镜像

由上面的分析可知，我们用来存储微程序的ROM的地址位宽是16，数据位宽是24。由于我们的指令周期最大为： $2^4 = 16$ ，且前6个时钟周期为取指周期，后面12个时钟周期用来执行指令。例如，对于 MOV C, 5 这个汇编语句，编译后的机器码为 0x84（二地址指令第7为1，MOV 对应 OP=0b000，再看寻址方式：目的操作数是立即数，因此为 0b00，原操作数为寄存器寻址，为 0b01，拼起来为 0b10000100=0x84），所以微程序ROM的16位地址从 0x8400 开始直到 0x84FF 均为该 MOV 指令所对应的区域，其中 0x8400~0x840f 为 PSW=0x0=0b0000 时对应的区域，其值应为：



图中蓝色底色的部分是取指令周期，红色虚线框部分是执行指令周期。该指令只需一个指令周期就能完成，硬件上设置C寄存器为写 DST-W=1，SRC寄存器为读 SRC-OUT，对应微程序ROM的数据第10位1，低4位为4：0b0000000000000010000000100 = 0x000404。接着一个时钟周期需要输出一个给周期计数器清零的控制，即 CYC_CLEAR=1 对应微程序数据第22为1，也就是对应二进制数 0b010000000000000000000000 = 0x40000。

至此，搭建一个图灵完备的8bitCPU所需要的理论知识介绍完毕！

愿你能在“从与非门到CPU的道路”上看到自己的坚持、努力和不平凡的创造力！